Literature Survey, Research Gap and Consistency Check

Project: Self-Healing Beyond Liveness: Model-Aware Recovery, Delivery and Failure Prediction for ML Forecasting Services on Kubernetes. Summaries come from each paper's abstract and first pages. Years marked ~ are inferred from the PDF and must be verified on IEEE Xplore before citing.

#	Paper	Authors	Year	What it states	Research gap	How our project differs
1	AI-Augmented Chaos Testing: Toward Intelligent Resilience Engineering in Cloud-Native Systems	S. Nigam, S. Patel	2025	GNN criticality analysis + transformer anomaly forecasting + RL to orchestrate chaos experiments, with a digital-twin 'Resilience Mirror'; up to 46% fewer SLO violations and 38% lower MTTR on Kubernetes microservices.	Generic microservice/IoT/GPU workloads; injects faults but does not treat ML model state (loaded/correct) as part of health or recovery.	We inject ML-specific faults (corrupt model = silent NaN output, OOM during model load, hang) on a forecasting service and add model-aware recovery.
2	A Comparative Study of Automatic Migration from Monolith App to Microservices	J. Zairi, A. Kenzi	~2025	Systematic review of 20 studies on automatic monolith decomposition; proposes a framework and evaluation criteria.	Review only: no deployed system, no runtime performance or fault-recovery data.	We migrate a real SARIMA+LSTM app and measure latency, recovery and failures for monolith vs Compose vs Kubernetes.
3	A Predictive Elastic Scaling Method for Kubernetes Based on Load Forecasting	L. Li	~2025	PathFormer forecasts CPU utilisation; replica counts are pre-computed and applied through Prometheus + the K8s API; outperforms native HPA.	Autoscaling for generic load; assumes a new pod is instantly useful (ignores model-load/warm-up time).	Our scaling experiment (secondary) accounts for model warm-up lead time and compares against HPA baselines.
4	A Stacking Ensemble ML-Based Failure Prediction Model for Optical Networks with Imbalanced Data	Z. Sun et al.	~2024	Stacking ensemble predicts failures when only 5.3% of samples are failures; F1 = 98.97%.	Optical-network telemetry; not containers or ML services.	Supports our handling of rare failures (class weights/SMOTE, report F1); we predict container/model failures from pod + forecast-quality signals.
5	AdaptiPID: Model-Aware Adaptive Gain Tuning with Anti-Windup Compensation for Robotic Manipulator Control	Y. Zhou	~2026	Adaptive PID with anti-windup and joint-coupling compensation for robot arms.	Unrelated domain (robot control).	Not relevant; excluded from related work.
6	Architecture-based Failure Prediction via LSTM and Bayesian Network in Service-oriented Systems	X. Wu et al.	~2022	LSTM predicts component reliability time series; a Bayesian network infers impending failures; +14% accuracy.	Component reliability series only; no output-quality or model-state signals; no recovery action.	Our predictor fuses pod telemetry with forecast-quality signals and triggers proactive replacement through the operator.
7	Autoencoder-based Anomaly Detection in Microservices using Distributed Tracing (AnoTraceAE)	See PDF header	2024	Convolutional autoencoder over span/trace embeddings; evaluated on TrainTicket.	Detection only; benchmark demo app; no ML-model health, no recovery.	We detect model-level failures (silent bad forecasts) and close the loop with recovery.
8	Chaos Engineering based Kubernetes Pod Rescheduling through Deep Sets and Reinforcement Learning	M. Zaccarini, F. Poltronieri, D. Borsatti, W. Cerroni, L. Foschini et al.	~2025	TELKA scheduler: chaos engineering + RL + digital twin to reallocate pods evicted by faults; Deep Sets removes the fixed-node-count limit.	Pod placement after node faults; not application-level correctness or model loading.	We study application-level silent failures and model-load recovery, not scheduling.
9	Deployment of Seasonal Autoregressive Integrated Moving Average (SARIMA) Models for Network Reliability Prediction	D. Singh	2024	Step-by-step deployment of SARIMA to forecast short-term network reliability.	Model use only; no containerisation, probes, recovery or operational metrics.	We deploy SARIMA and LSTM as services and study their operational reliability (MTTR, availability).
10	Evaluating operational readiness using chaos engineering simulations on Kubernetes architecture in Big Data	G. Siwach, A. Haridas	~2022	Random pod termination in a Big Data K8s setup; infers pod recovery time.	Only pod kills; no ML model state, no silent faults, no statistical experiment design.	Four-fault taxonomy, repeated trials, and a DoE/RSM MTTR model with model-load decomposition.
11	Is it Worth Migrating a Monolith to Microservices? An Experience Report on Performance, Availability and Energy Usage	V. Berry et al.	~2024	Migrates a web app; MSA wins without replication, monolith wins when the whole app is replicated; also studies energy.	Generic web app; no ML models or fault-recovery behaviour.	Same A/B/C comparison on an ML app, extended with fault injection and model-aware recovery.

#	Paper	Authors	Year	What it states	Research gap	How our project differs
12	Kube-CC: Building an Experiment Education Platform for Big Data Courses Based on Kubernetes	F. Xue et al.	~2023	K8s + Go platform for one-click Big Data cluster deployment and recovery in teaching labs.	Education platform; no resilience measurements.	Background on Kubernetes use only.
13	MicroMILTS: Fault Location for Microservices Based on Mutual Information and LSTM Autoencoder	L. Yang et al.	~2022	BIRCH anomaly detection, then LSTM-autoencoder reconstruction deviations + mutual information to locate the faulty service.	Localises faults from infrastructure metrics; no model-quality signals; no recovery.	We add forecast-quality signals and automatic recovery.
14	Multi-Level Ensemble Anomaly Detection for Microservices: Statistical Analysis, Isolation Forest and Adaptive Thresholding	See PDF header	~2025	Prometheus metrics through statistical profiling + Isolation Forest + adaptive thresholds, with a dashboard; ERP-like testbed.	Metric-only detection on a synthetic ERP; no ML inference service, no healing.	Isolation Forest is used on pod telemetry fused with output-quality signals and wired into recovery.
15	Recall-Prioritized Anomaly Detection in Cloud Microservices using Unsupervised Transformer Autoencoders	A. C. et al.	~2025	LogBERT-Lite transformer autoencoder on OpenStack logs; high recall (8 of 11 injected faults) at 4.37% precision.	Log-based, low precision; no remediation.	We target silent ML faults and tie detection to recovery with a measured lead time.
16	Research on Constraint-Aware Fault Recovery Diagnosis Decision Method for Offline Edge Kubernetes Scenarios	Z. Wei, K. Bai	~2025	CAR: Repair / Degrade / Rebuild decision tree; MTTR 127 -> 34 min in an offline ARM64 case study.	Manual-style diagnosis for constrained edge clusters; no ML service.	Our degrade tier (LSTM -> SARIMA -> climatology) is an automated, ML-service-specific analogue.
17	Residual Artifact Governance With ML-Enabled Garbage Collection Prediction in Kubernetes-Based 5G/6G Service-Based Architecture	L.-M. Tufeanu et al.	2026	Master-agent garbage collection with an ML predictor of time-to-collapse from residual artifacts.	Telecom core network; residual-artifact leaks, not ML service correctness.	Same idea of ML early warning, applied to forecasting-service failures.
18	Robust Hybrid Autoencoder-LSTM Framework for Real-Time Anomaly Detection in Multi-Cloud Environments	S. Oviya et al.	~2025	Autoencoder + LSTM combined score; 94.2% accuracy and 5.6% false positives on CICIDS2017.	Network-intrusion datasets; no operational recovery.	LSTM autoencoder is used as a comparison detector on our own fault traces.
19	Service Level Objective (SLO) Observability with Splunk and Dynatrace in Microservices	V. P. K. Gummadi, L. S. Chilamkurti	~2025	SLO/SLI-centred observability, error budgets, reduced alert fatigue.	Practice/commercial-tool paper; no experiments on ML services.	We implement SLOs, burn-rate alerts and error budgets for a forecasting service with Prometheus/Grafana.
20	The Multi-Modal Anomaly Detection Method for Microservices Based on VGAE-Transformer (VTMAD)	C. Shao, X. Zhou et al.	~2025	VGAE + Transformer on multimodal data; reconstruction error on the adjacency matrix.	Heavy detection model on public datasets; no healing.	Lightweight, deployable detector plugged into an operator.
21	TraceDAE: Trace-Based Anomaly Detection in Microservice Systems via Dual Autoencoder	J. Li, S. Ying, T. Li et al.	2025	Service Trace Graph; GAT structure autoencoder + LSTM attribute autoencoder; detects response and invocation anomalies.	Trace anomalies only, offline datasets; no model-health or recovery.	Detection feeds real recovery and is evaluated on MTTR and failed requests.

Consistency check: where papers and our implementation may conflict

Topic	Overlapping paper(s)	Conflict with our claim?	What we do / how to word it
Chaos / fault injection + MTTR	AI-Augmented Chaos Testing (2025); Big Data chaos readiness; Pod Rescheduling (TELKA)	Partial. Pod kill + recovery time is already done; AI-driven adaptive chaos exists.	Do NOT claim chaos testing or adaptive chaos as novel. Our contribution is the ML-specific fault taxonomy (silent corrupt output, model-load OOM, hang) and model-aware recovery. State that our fault scenarios are static (limitation).
Graceful degradation tiers	Constraint-Aware Recovery (Repair / Degrade / Rebuild)	Partial. A Degrade strategy already exists in Kubernetes recovery.	Reword novelty 5: ML-service-specific automated tiers (LSTM -> SARIMA -> climatology -> cache) with a measured availability vs forecast-error trade-off; cite CAR as related.
Anomaly / failure prediction	Multi-Level Ensemble (Isolation Forest on Prometheus); TraceDAE; VTMAD; MicroMILTS; Hybrid AE-LSTM; Architecture-based LSTM+BN	Yes if we claim the detector itself. No if we claim the signals and closed loop.	Do NOT claim Isolation Forest or LSTM-AE as novel. Claim: fusing forecast-output-quality signals with pod telemetry and triggering recovery, with lead time and false-positive rate measured.
Predictive autoscaling	Predictive Elastic Scaling (PathFormer); earlier SARIMA PBA, DQN, Fast-response HPA	Yes if headline. Autoscaling is crowded.	Keep as a secondary experiment (warm-up-aware lead time) with HPA baselines; not a headline novelty.
Monolith vs microservices	Berry et al. experience report; Zairi & Kenzi review	Partial. Performance/availability comparison already published for a web app.	Do NOT claim migration comparison as novel; present A/B/C as the experimental platform. Novelty is the ML workload plus fault behaviour.
SLO / error budgets	SLO Observability with Splunk and Dynatrace	No. Practice paper, no ML service experiments.	Implement SLO burn-rate alerts for the forecasting service and cite as related practice.
Silent failures (HTTP 200, NaN output)	None of the 21 papers	No overlap found in these 21 papers.	Strongest gap, and already reproduced: corrupt fault is never recovered by default Kubernetes probes. Say "to our knowledge".

Consistency check: implementation status vs paper claims

Item	Status in our implementation	Risk / action
Config A / B / C platform	Built: monolith, Docker Compose microservices, Kubernetes (Minikube) with Prometheus + Grafana.	OK.
Fault taxonomy	Implemented: crash, hang, oom, corrupt (/fault endpoint).	crash/oom are partly recovered by gunicorn respawning its worker, not only by Kubernetes. Report this explicitly or run with one-process-per-container to isolate.
Results so far	Only 11 trials, all config C (crash 12-14 s, hang 62-67 s, oom ~21.5 s, corrupt never recovers). Batch stopped early; B and A not run.	Do not quote numbers until the batch is rerun to completion with >= 10 trials per cell.
Novelty 1 semantic probe	NOT yet implemented (Phase 4).	Claim only after Phase 4 results.
Novelty 2 DoE/RSM MTTR model	NOT yet implemented (Phase 7).	Needs central composite design runs.
Novelty 3 failure predictor	NOT yet implemented (Phase 6).	Needs labelled fault traces first.
Novelty 4 model-quality rollout gate	NOT yet implemented (Phase 5).	Argo/CI work pending.
Novelty 5 operator + tiered fallback	Partially: auto picks LSTM then SARIMA. No climatology/cache tier or operator yet.	Phase 4.
Original proposal assumption	Proposal said SARIMA models are ~120 MB and slow to reload. Measured: SARIMA loads in ~1.3-1.6 s each; LSTM 1.9-10.8 s (TF warm-up).	Conflict with proposal text: update wording; the cost is mostly LSTM/TensorFlow warm-up and image size (inference image 2.8 GB).
LSTM in original app	Original /lstm_result used a seasonal-repeat stand-in; DevOps version runs the trained .h5 models.	Numbers on the LSTM page changed; document this.
OpenTelemetry tracing	Not implemented.	Either implement or drop from claims (TraceDAE/AnoTraceAE use tracing).
Years / authors in survey	Some years inferred; two author lists unreadable from page 1.	Verify on IEEE Xplore.